

실습을 통해 라마인덱스 라이브러리의 사용법을 간단히 알아보면서, LLM 오케스트레이션 라이브러리가 LLM 애플리케이션 개발에 얼마나 큰 도움이 되는지 확인해 볼 수 있을 것이다.

여기서 llama-index==0.10.34는 텍스트 데이터 검색과 색인화를 위한 NLP 라이브러리이고, langchain-openai==0.1.6은 OpenAI의 GPT 모델과 상호작용하는 체인을 만드는 도구이며, openai==1.25.1은 OpenAI API를 통해 GPT 모델에 접근하고 활용할 수 있는 라이브러리이다.

```
!pip install datasets llama-index==0.10.34 langchain-openai==0.1.6 #  
nemoguardrails[openai]==0.8.0 openai==1.25.1 chromadb==0.5.0 wandb==0.16.6 #  
llama-index-callbacks-wandb==0.1.2 -qqq  
  
Preparing metadata (setup.py) ... done  
----- 41.9/41.9 kB 1.5 MB/s eta 0:00:00  
----- 67.3/67.3 kB 2.6 MB/s eta 0:00:00  
  
Installing build dependencies ... done  
Getting requirements to build wheel ... done  
Preparing metadata (pyproject.toml) ... done  
----- 50.4/50.4 kB 3.1 MB/s eta 0:00:00  
----- 2.5/2.5 MB 49.8 MB/s eta 0:00:00
```

datasets 라이브러리의 load_dataset 함수에 데이터셋 이름인 klue와 서브셋 이름인 mrc를 입력하면 허깅페이스 데이터셋 허브에 KLUE MRC 데이터셋을 다운로드한다. 그리고 dataset[0]을 통해 내려받은 데이터셋의 첫 번째 데이터를 확인한다.

```
import os
from datasets import load_dataset

os.environ['OPENAI_API_KEY'] = "sk-..."

dataset = load_dataset('klue', 'mrcc', split='train')
dataset[0]
```

/usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_token.py:89: UserWarning:
The secret 'HF_TOKEN' does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.

warnings.warn(
READMEmd: 100% ██████████ 22.5k/22.5k [00:00<00:00, 693kB/s]
train-00000-of-00001.parquet: 100% ██████████ 21.4M/21.4M [00:00<00:00, 31.4MB/s]
validation-00000-of-00001.parquet: 100% ██████████ 8.68M/8.68M [00:00<00:00, 42.1MB/s]
Generating train split: 100% ██████████ 17554/17554 [00:00<00:00, 44895.95 examples/s]
Generating validation split: 100% ██████████ 5841/5841 [00:00<00:00, 32067.38 examples/s]

```
{  
  'title': '제2호 중마 시작 ... 중부는 이날 말부터',  
  'context': '올들어 중마가 17일 제주도에 시작됐다. 서울 등 중부지방은 예년보다 사나운 정도 높은 이날 말게 중마가 시작될 전망이다. 17일 기상청에 따르면 제주도 남쪽 먼바다에 있는 장마전선의 영향으로 이날 제주도 산간 및 내륙지역에 호우주의가 내려지면서 곳곳에 100mm에 육박하는 많은 비가 내렸다. 제주의 장마는 평년보다 2~3일, 지난해보다는 하루 일찍 시작됐다. 장마는 고온다습한 북태평양 기단과 한랭 습윤한 오호츠크해 기단이 만나 형성 되는 장마전선에서 내리는 비를 뜻한다. 장마전선은 18일 제주도 먼 남쪽 해상으로 내려갔다가 20일께 다시 북상해 전남 남해안까지 영향을 줄 것으로 보인다. 이에 따라 20~21일 남부지방에도 예년보다 사흘 정도 중마가 일찍 찾아올 전망이다. 그러나 장마전선을 밀어올리는 북태평양 고기압 세력이 약해 서울 등 중부지방은 평년보다 사나울까한 늦은 이날 말부터 중마가 시작될 것이라는 게 기상청의 설명이다. 장마전선은 이후 한 달가량 한반도 중남부를 오르내리며 곳곳에 비를 뿌릴 전망이다. 최근 30년간 평균치에 따르면 중부지방의 장마 시작일은 6월24~25일일 것으로 장마기간은 32일, 강수량수는 17.2일이다. 기상청은 올해 장마기간의 평균 강수량이 350~400mm로 평년과 비슷하거나 적을 것으로 내다봤다. 라벨 월드컵 한국과 러시아의 경기가 열리는 18일 오전 서울은 대체로 구름이 많이 끼지만 비는 오지 않을 것으로 예상돼 거리 등행에는 지장이 없을 전망이다.'  
  'news_category': '종합',  
  'source': 'hankyung',  
  'guid': 'klue-mrc-v1_train_12759',  
  'is_impossible': False,  
  'question_type': 1,  
  'question': '북태평양 기단과 오호츠크해 기단이 만나 국내에 머무르는 기간은?',  
  'answers': {'answer_start': [478, 478], 'text': ['한 달가량', '한 달']}]
```

3. 실습 데이터 중 앞부분 100개만 뽑아 임베딩 벡터로 만들고 벡터 데이터베이스에 저장

dataset[:100]['context']에서 첫 100개의 맥락 데이터를 추출하고 라마인덱스의 Document 클래스의 text인자에 맥락(context) 데이터를 전달해 라마인덱스가 어떤 텍스트를 임베딩 벡터로 변환할지 지정한다.

라마인덱스는 기본 임베딩 모델로 OpenAI의 text-embedding-ada-002 모델을 사용하고 기본 벡터 데이터베이스로 인메모리 방식의 벡터 데이터베이스를 사용한다.

```
[3] from llama_index.core import Document, VectorStoreIndex

text_list = dataset[:100]['context']
documents = [Document(text=t) for t in text_list]

# 인덱스 만들기
index = VectorStoreIndex.from_documents(documents)
```

[nltk_data] Downloading package punkt_tab to
[nltk_data] /usr/local/lib/python3.10/dist-
[nltk_data] packages/llama_index/core/_static/nltk_cache,...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.

VectorStoreIndex 클래스의 from_documents() 메서드를 사용하면 우리가 Document 클래스로 생성한 documents를 입력으로 해서 라마인덱스가 내부적으로 텍스트를 임베딩 벡터로 변환해 인메모리 벡터 데이터베이스에 저장한다.

4. 100개의 기사 본문 데이터에서 질문과 가까운 기사 찾기

이제 100개의 기사 본문을 저장한 벡터 데이터베이스에서 코드로 예시의 질문('북태평양 기단과...')과 유사한 기사 본문을 찾아보자. 기사 본문을 저장한 인덱스 (index)를 벡터 검색에 사용할 수 있도록 as_retriever 메서드로 검색 엔진으로 변환한다. 이때 가장 가까운 5개의 기사를 반환하도록 similarity_top_k 인자에 5를 전달했다.

검색 엔진(retrieval_engine)에 retrieve 메서드로 찾으려는 질문을 입력으로 넣으면 입력한 질문과 가장 유사한 5개의 기사 본문을 찾아 반환한다.

```
[4] print(dataset[0]['question']) # 북태평양 기단과 오후초크해 기단이 만나 국내에 머무르는 기간은?

retrieval_engine = index.as_retriever(similarity_top_k=5, verbose=True)
response = retrieval_engine.retrieve(dataset[0]['question'])

print(len(response)) # 출력 결과: 5
print(response[0].node.text)
```

북태평양 기단과 오후초크해 기단이 만나 국내에 머무르는 기간은?
4
올여름 장마가 17일 제주도에서 시작했다. 서울 등 중부지방은 예년보다 사나흘 정도 늦은 이달 말께 장마가 시작될 전망이다. 17일 기상청에 따르!

하지만 내가 실습한 결과에서는 5가 아닌 4가 출력되지만 찾은 기사 본문 내용들은 책에 나온 내용과 같았다.

5개 출력되지 않은 이유를 추측해 보자면, 검색된 기사들의 유사도가 충분히 높지 않기 때문에 4로 출력된 거 같다.

5. 라마인덱스를 활용해 검색 증강 생성 수행하기

이번에는 검색한 본문을 활용해 LLM의 답변까지 생성해 봤다. 아래 코드를 실행하면 인덱스를 as_query_engine 메서드를 통해 쿼리 엔진으로 변환하고, query 메서드에 질문을 입력하면 질문과 관련된 기사 본문을 찾아 프롬프트에 추가하고 LLM의 답변까지 생성한다.

라마인덱스는 OpenAI의 gpt-3.5-turbo를 기본 언어 모델로 사용한다.

```
✓ 2초 [13] query_engine = index.as_query_engine(similarity_top_k=1)
      response = query_engine.query( dataset[0]['question'] )
      print(response) # 장마전선에서 북태평양 기단과 오호츠크해 기단이 만나 국내에 머무르는 기간은 한 달 정도입니다.
```

☞ 장마전선에서 내리는 비를 뜻하는 장마는 북태평양 기단과 오호츠크해 기단이 만나 국내에 한 달가량 머무르게 됩니다.

아래 출력 결과를 확인하면 질문에 대한 답이 '한 달가량'이라고 정확히 답한 것을 확인할 수 있다.

6. 라마인덱스 내부에서 검색 증강 생성을 수행하는 과정

앞서 RAG에 대해 설명하면서 다양한 구성요소와 과정이 필요하다고 했는데, 라마인덱스를 사용하면 5번과 같이 단 두 줄의 코드만으로 유사한 텍스트를 검색하고 생성하는 모든 과정을 수행할 수 있다. 중간의 모든 과정을 내부에서 알아서 처리하기 때문에 편하기도 하지만, 정확히 어떻게 작동하는지 파악하기 어렵다는 단점도 있다.

```
✓ 2초 [78] from llama_index.core import ( VectorStoreIndex, get_response_synthesizer, )

      from llama_index.core.retrievers import VectorIndexRetriever
      from llama_index.core.query_engine import RetrieverQueryEngine
      from llama_index.core.postprocessor import SimilarityPostprocessor

      # 검색을 위한 Retriever 생성
      retriever = VectorIndexRetriever(
          index=index,
          similarity_top_k=1
      )

      # 검색 결과를 질문과 결합하는 synthesizer
      response_synthesizer = get_response_synthesizer()

      # 위의 두 요소를 결합해 쿼리 엔진 생성
      query_engine = RetrieverQueryEngine(
          retriever=retriever,
          response_synthesizer=response_synthesizer,
          node_postprocessors=[SimilarityPostprocessor(similarity_cutoff=0.7)],
      )

      # RAG 수행
      response = query_engine.query( "북태평양 기단과 오호츠크해 기단이 만나 국내에 머무르는 기간은?" )
      print(response)
      #장마전선에서 북태평양 기단과 오호츠크해 기단이 만나 국내에 머무르는 기간은 한 달가량입니다.
```

☞ 장마전선에서 내리는 비를 뜻하는 장마는 고온다습한 북태평양 기단과 한랭 습윤한 오호츠크해 기단이 만나 형성되며, 한 달가량 국내에 머무르게 됩니다.

따라서 내부적으로는 위와 같이 세 단계를 거쳐 RAG를 수행한다. 먼저 VectorIndexRetriever 클래스를 사용해 벡터 데이터베이스에서 검색하는 retriever를 만든다. 검색 결과를 사용자의 요청과 통합하기 위해 get_response_synthesizer() 함수를 사용해 프롬프트를 통합할 때 사용할 response_synthesizer를 만든다. 마지막으로 RetrieverQueryEngine 클래스에 앞서 생성한 retriever와 response_synthesizer를 전달해 검색 증강 생성을한 번에 수행하는 query_engine을 생성한다. 이때 SimilarityPostprocessor와 같은 클래스를 사용해 질문과 유사도가 낮은 경우는 필터링하도록 설정할 수 있다. 위 코드에서는 유사도가 0.7 이하인 문서는 필터링하도록 설정했다.

특이점 - 같은 질문이라도 다른 답변을 출력함

아래 코드에서 `dataset[0]['question']` 부분을 보면, 다 같은 질문을 입력했음에도 불구하고 매번 다른 답변을 출력하는 경우가 발생할 수 있다. 이 현상은 LLM이 동작하는 방식과 관련이 있다. GPT-3.5-turbo와 같은 언어 모델은 같은 질문을 입력해도 그 질문을 해석하고 답변을 생성하는 과정에서 어느 정도의 랜덤성을 포함한다.

이 랜덤성은 특히 온도(temperature) 설정에 따라 달라지며, 온도가 높을수록 더 다양한 답변을 생성할 가능성이 커진다.

또한, 질문에 관련된 본문이 여러 개일 경우, `similarity_top_k` 설정에 따라 인덱스에서 선택된 기사의 내용이 달라질 수 있다. 이로 인해 LLM이 참고하는 정보가 달라지고, 그 결과 다른 답변을 출력할 가능성이 생긴다.

```
[11] query_engine = index.as_query_engine(similarity_top_k=1)
      response = query_engine.query(dataset[0]['question'])
      print(response) # 장마전선에서 북태평양 기단과 오호츠크해 기단이 만나 국내에 머무르는 기간은 한 달 정도입니다.

장마전선에서 내리는 비를 뜻하는 장마는 북태평양 기단과 오호츠크해 기단이 만나 국내에 한 달가량 머무르게 됩니다.

[14] query_engine = index.as_query_engine(similarity_top_k=1)
      response = query_engine.query(dataset[0]['question'])
      print(response) # 장마전선에서 북태평양 기단과 오호츠크해 기단이 만나 국내에 머무르는 기간은 한 달 정도입니다.

장마전선에서 내리는 비를 뜻하는 장마는 북태평양 기단과 오호츠크해 기단이 만나 형성됩니다.

[17] query_engine = index.as_query_engine(similarity_top_k=1)
      response = query_engine.query(dataset[0]['question'])
      print(response) # 장마전선에서 북태평양 기단과 오호츠크해 기단이 만나 국내에 머무르는 기간은 한 달 정도입니다.

한 달
```

결론

지금까지 LLM에 생성에 필요한 데이터를 제공하는 RAG에 대해 알아보고 라마인덱스를 활용해 RAG를 구현하는 실습을 진행했다. 라마인덱스와 같은 LLM 오케스트레이션 도구를 활용하면 몇 줄의 코드만으로 RAG를 구현할 수 있다.